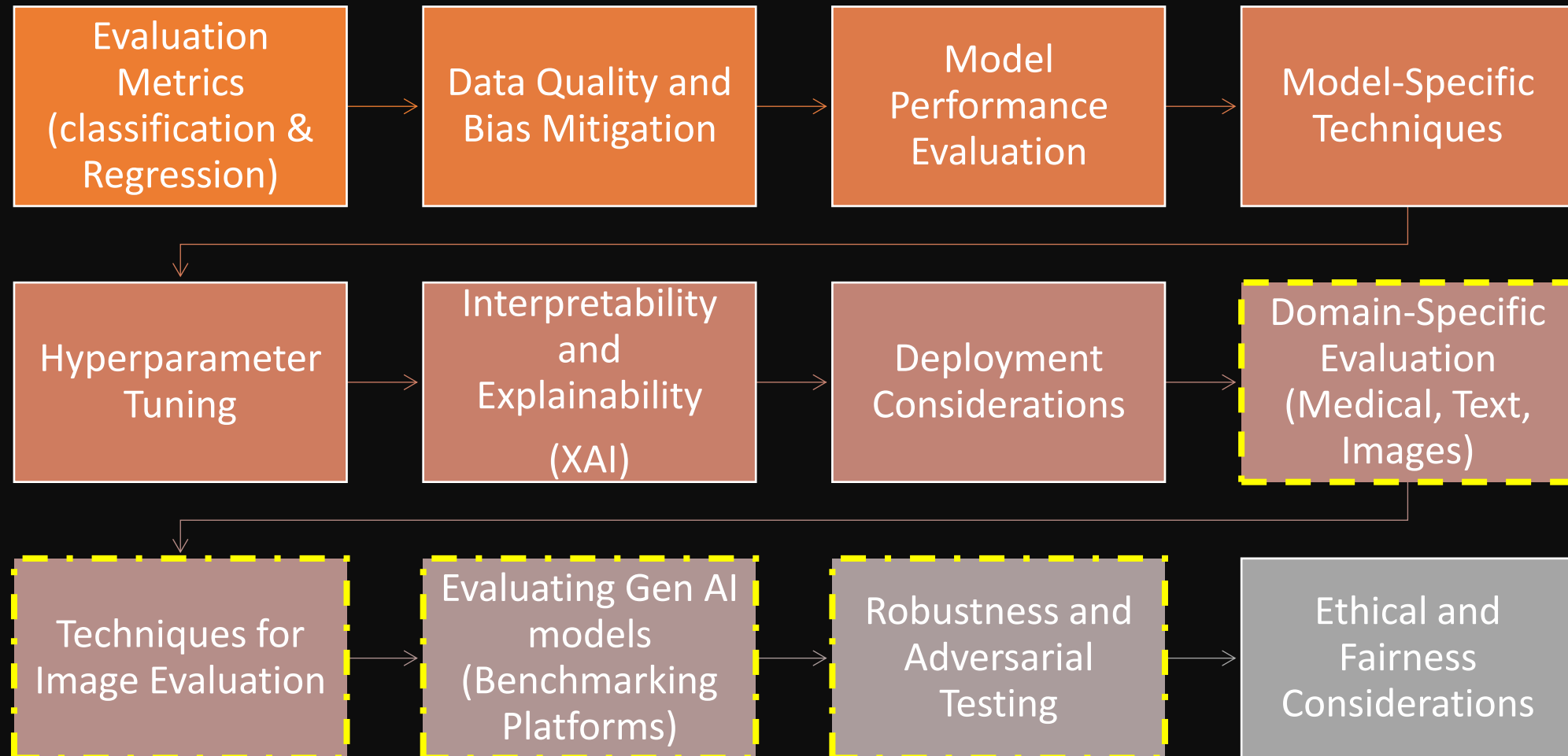


Evaluate AI Models

Classical models to
GenAI models
(LLMs, images)



Evaluating AI models



Evaluation Metrics

Classification

- Metrics such as
 - accuracy (proportion of correct predictions),
 - precision (true positives divided by the sum of true positives and false positives),
 - recall (true positives divided by the sum of true positives and false negatives),
 - F1-score (harmonic mean of precision and recall),
 - ROC-AUC score (area under the Receiver Operating Characteristic curve), and
 - confusion matrix (table showing true positives, false positives, true negatives, and false negatives).

Regression

- Metrics like
 - Mean Squared Error (MSE) (average squared difference between predicted values and actual values),
 - Mean Absolute Error (MAE) (average absolute difference between predicted values and actual values), and
 - R-squared (coefficient of determination) indicating how well the predictions approximate the real data points).

Data Preparation (splitters)

Splitting

Data Splitting

- Dividing the dataset into training (used for model training), validation (used for [model selection and hyperparameter tuning](#)), and test sets (used for final evaluation of the model).

Preprocessing

Data Preprocessing

- Techniques such as normalization (scaling numeric data to a standard range), standardization (transforming data to have zero mean and unit variance), handling missing values, and performing feature scaling (ensuring all features have a similar scale).

Model Performance Evaluation

Cross-Validation:

- Techniques like **K-fold cross-validation** (dividing data into K subsets and using each as validation while the rest as training data) and **Stratified K-fold cross-validation** (maintaining the percentage of samples for each class during cross-validation) to assess model performance and generalize well to new data.

Learning Curves:

- Plots showing the model's performance (e.g., error) on both the training and validation sets against the training set size, helping to diagnose underfitting or overfitting.

Bias-Variance Analysis:

- Understanding the trade-off between **bias** (error from erroneous assumptions) and **variance** (error from sensitivity to small fluctuations in the training set).

Model-Specific Techniques

Decision Trees and Random Forests:

- Analyzing **feature importance** (how much each feature contributes to the model's predictions) and visualizing individual **decision trees**.

Support Vector Machines (SVM):

- Evaluating **margin** (distance between the decision boundary and support vectors) and performance with different **kernels** (functions used to transform data into higher-dimensional space).

Neural Networks:

- Examining **training curves** (learning progress over epochs), analyzing **activation functions** (functions that determine the output of a neural node), and checking **gradient** (rate of change of a function concerning a variable) integrity.

Ensemble Methods:

- Comparing individual models within the ensemble to assess their contributions to overall prediction performance.

Hyperparameter Tuning

Grid Search

- Exhaustively searching for optimal hyperparameter combinations by evaluating model performance across all possible parameter values.

Random Search

- Sampling hyperparameter combinations randomly from a defined search space to efficiently find good configurations.

Bayesian Optimization

- Using probabilistic models to search for the optimal set of hyperparameters based on past evaluations.

Model Interpretability

Feature Importance

- Determining which features most influence the model's predictions.

Partial Dependence Plots (PDP)

- Showing how a feature affects predictions while marginalizing the effects of all other features.

SHAP (SHapley Additive exPlanations) values

- Providing explanations for individual predictions based on contributions of each feature.

Deployment Considerations

Scalability: Testing the model's performance with larger datasets or in real-time prediction scenarios.

Latency: Evaluating the time it takes for the model to generate predictions, critical for applications requiring quick responses.

Monitoring: Setting up systems to continuously monitor the model's performance in production environments to detect issues or drift.

Domain-Specific Evaluation

Medical Imaging: Using metrics like sensitivity, specificity, and the Dice coefficient to evaluate performance in medical image analysis.

Natural Language Processing (NLP): Evaluating models using metrics such as BLEU score (for machine translation), ROUGE score (for summarization), and perplexity (for language modeling).

evaluating generative models



Inception Score:

Measures the quality and diversity of generated images.



Precision and Recall:

Can be adapted for generative models to evaluate how well the generated samples match the distribution of the real data.



Structural Similarity Index (SSIM):

Measures the similarity between two images, considering luminance, contrast, and structure,



Fréchet Inception Distance (FID):

Evaluates the similarity between the distribution of generated images and real images in feature space.



Mean Square Error (MSE):

Measures the pixel-wise difference between generated images and real images, although this is less



Kernel Inception Distance (KID):

Another metric that evaluates the similarity between distributions of generated and real images,

Ethical and Fairness Considerations

Bias Detection:

- Identifying biases in predictions that might disproportionately affect certain demographic groups.

Fairness Metrics:

- Assessing fairness across different subgroups to ensure equitable outcomes.

Sequence Labelling Metrics

Used for tasks like named entity recognition (NER) or part-of-speech tagging.

- **Precision, Recall, F1 Score:** Computed for each label and then averaged.
- **Token Accuracy:** The ratio of correctly predicted tokens to the total number of tokens.
- **Sequence Accuracy:** The ratio of correctly predicted sequences to the total number of sequences.



Language Generation Metrics

- **BLEU (Bilingual Evaluation Understudy Score)**: Measures the precision of n-grams in the generated text compared to a reference text.
- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)**: Measures the recall of n-grams, typically used in summarization.
 - **ROUGE-N**: Measures the overlap of n-grams.
 - **ROUGE-L**: Measures the longest common subsequence.
- **METEOR (Metric for Evaluation of Translation with Explicit ORdering)**: Considers synonyms, stemming, and word order.
- **CIDEr (Consensus-based Image Description Evaluation)**: Measures the consensus between generated and reference sentences, often used in image captioning.
- **TER (Translation Edit Rate)**: Measures the number of edits required to change the generated text into the reference text.
- **Perplexity**: Measures the uncertainty in predicting the next word in a sequence, commonly used in language models.

General-purpose Metrics

Log-Likelihood: The likelihood of the observed data given the model parameters.

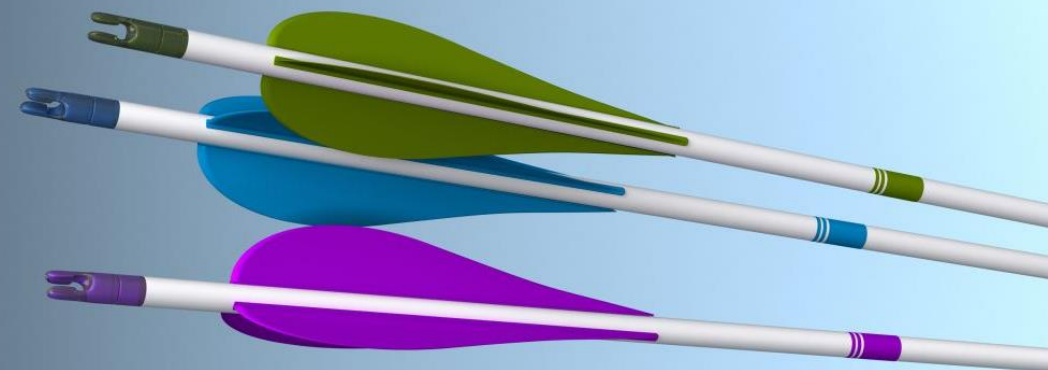
Mean Reciprocal Rank (MRR): Used in information retrieval, measures the rank of the first relevant document.

Mean Average Precision (MAP): The average precision across multiple queries or samples.

NDCG (Normalized Discounted Cumulative Gain): Measures the usefulness of a document based on its position in the result list, used in ranking tasks.

Robustness and Fairness Metrics

- **Adversarial Accuracy:** Measures the model's accuracy when adversarial examples are introduced.
- **Fairness Metrics:** Measure biases across different groups (e.g., demographic parity, equal opportunity).





Human Evaluation Metrics

- **Fluency:** Measures how fluent and grammatically correct the generated text is.
- **Coherence:** Measures how logically consistent and connected the text is.
- **Relevance:** Measures how relevant the generated text is to the given prompt or context.
- **Engagement:** Measures how engaging or interesting the text is to the reader.

Task-specific Metrics

01

Word Error Rate (WER):
Used in speech recognition to measure the number of errors in the transcribed text.

02

Edit Distance (Levenshtein Distance): Measures the minimum number of edits required to change one text into another, useful in text similarity tasks.

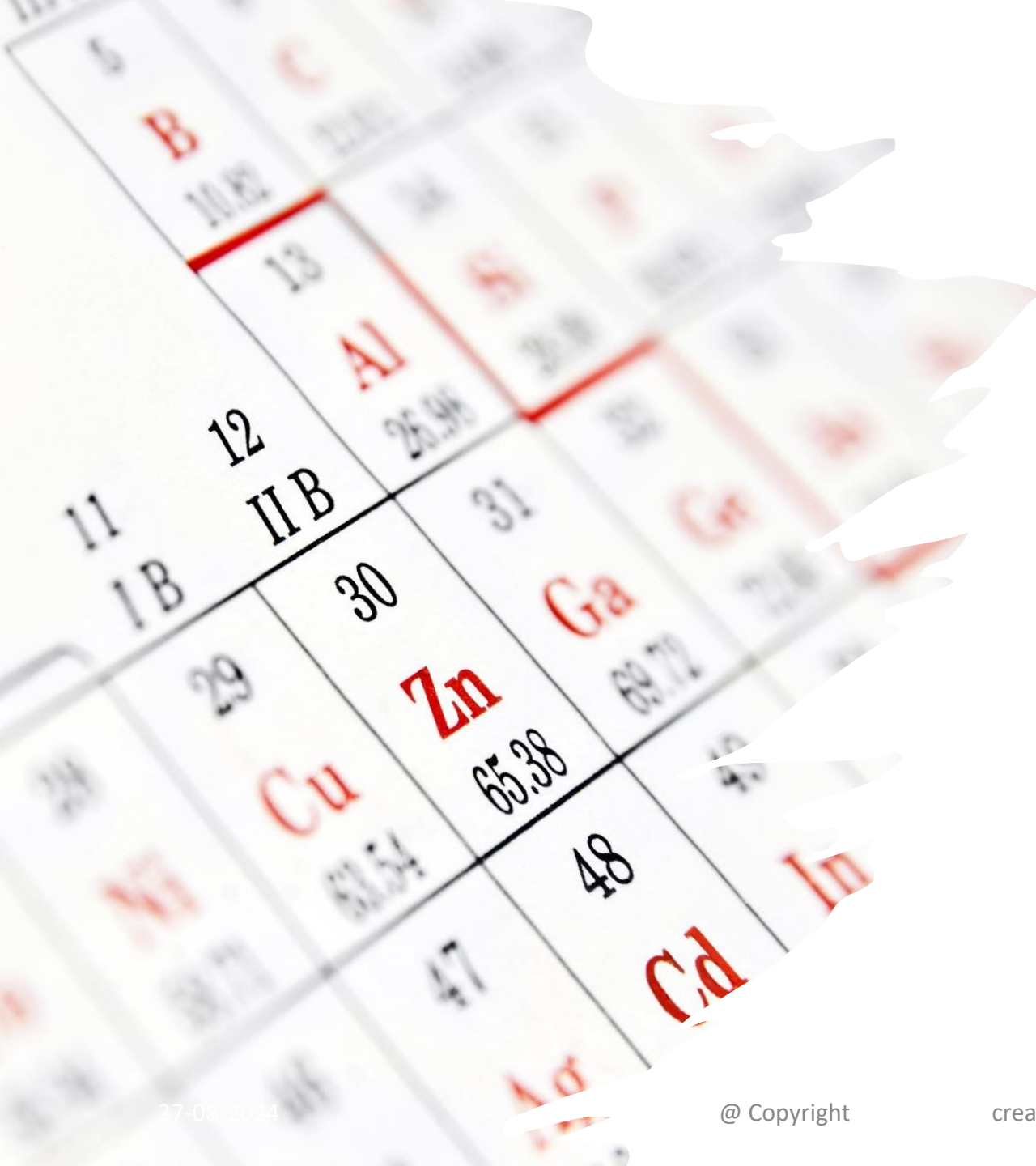
03

Extractive Metrics: For summarization tasks, measures such as compression ratio and summary length.

A close-up, high-contrast photograph of a ruler. The ruler is dark with white markings. A needle points to the number 5. The background is blurred, showing more of the ruler and the number 4.

Let us detail out

Classification Metrics



Classification Metrics

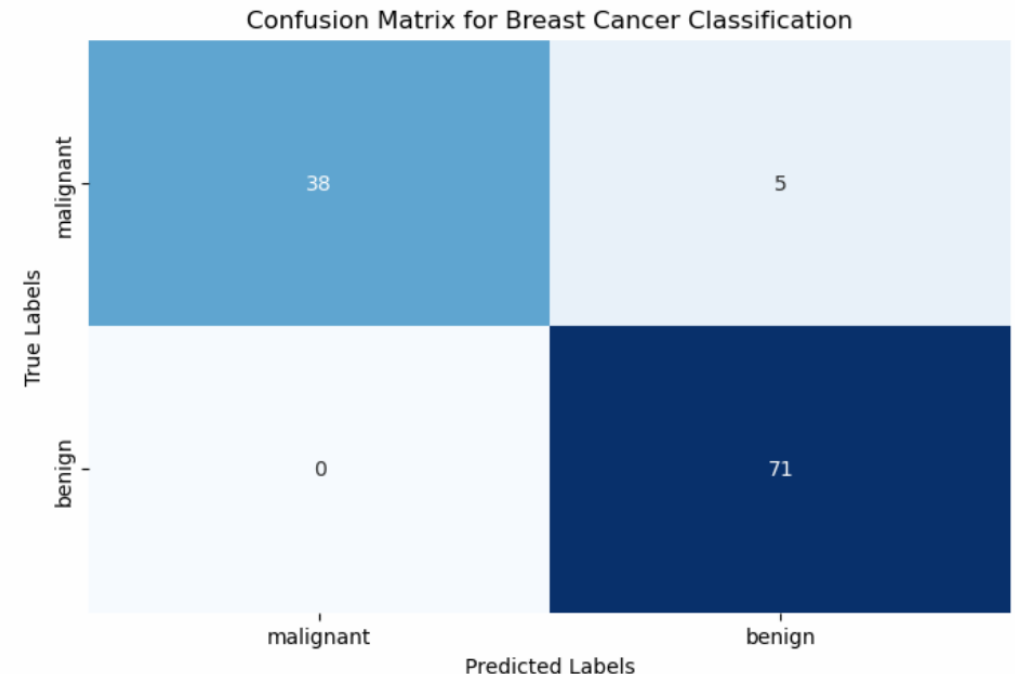
- **Accuracy:** ratio of correctly predicted labels to the total number of predictions.
- **Precision:** ratio of true positive predictions to the total number of positive predictions.
- **Recall:** ratio of true positive predictions to the total number of actual positives.
- **F1 Score:** harmonic mean of precision and recall.
- **ROC-AUC (Receiver Operating Characteristic - Area Under Curve):** Measures the model's ability to distinguish between classes.
- **Confusion Matrix:** A table showing the true positives, true negatives, false positives, and false negatives.

Confusion matrix

- $y_{\text{test}} = [1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 2, 2, 2, 3, 3, 3, 3, 3, 3]$
- $y_{\text{pred}} = [1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 2, 1, 0, 3, 3, 3, 3, 1, 2, 3]$

Confusion Matrix :

```
array([[4, 2, 0, 0],  
       [1, 3, 0, 0],  
       [1, 1, 1, 0],  
       [0, 1, 1, 5]])
```





		Predicted Class	
		Class = 0	Class = 1
Actual Class	Class = 0	f_{00} TN	f_{01} FP
	Class = 1	f_{10} FN	f_{11} TP

Norms on confusion matrix

0 - Negative (E.g. does not have diabetes, NO)

1 - Positive (E.g. has diabetes, or YES)

f_{00} - True Negative (TN)

f_{11} - True Positive (TP)

f_{01} - False Positive (FP) ..Also called type-I

f_{10} - False Negative (FN) ..Also called type-II

correct predictions $\rightarrow f_{00} + f_{11}$ || Incorrect predictions $\rightarrow f_{01} + f_{10}$

accuracy = correct predictions / total predictions

$$= (f_{00} + f_{11}) / (f_{00} + f_{10} + f_{01} + f_{11})$$

error rate = incorrect predictions / total predictions

$$= (f_{10} + f_{01}) / (f_{00} + f_{10} + f_{01} + f_{11})$$

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Classification Report

	precision	recall	f1-score	support
0	1.00	0.88	0.94	43
1	0.93	1.00	0.97	71
accuracy			0.96	114
macro avg	0.97	0.94	0.95	114
weighted avg	0.96	0.96	0.96	114

Precision: measures the accuracy of positive predictions made by the model.

Recall (Sensitivity or True Positive Rate): measures the ability of the model to capture all positive instances in the dataset.

F1 score : harmonic mean of precision and recall. It provides a balanced measure that considers both false positives and false negatives.

Support represents the number of occurrences of each class in the true dataset

Accuracy measures the overall correctness of predictions made by the model.

Recall

- also known as **Sensitivity** or **True Positive Rate**, evaluates the ability of a model to capture all **positive** instances in a dataset.
- It is particularly relevant in scenarios where the cost of false negatives (predicting negative when the actual class is positive) is relatively high.

$$\text{Recall} = \frac{TP}{TP + FN}$$

		PREDICTED		
		No default (0 or N - Negative)	Default (1 - P - Positive)	
ACTUAL	No default (0 or N - Negative)	TN 4	Type I error 2 FP	6
	Default (1 - P - Positive)	FN 1 Type II error	3 TP	4
		5	5	0

for - breast cancer data

Classes of data

- Class 0: Benign tumors (Negative class)
- Class 1: Malignant tumors (Positive class)

For Class 0 (Benign tumors)

- Recall (0) - measures the ability of the model to correctly identify instances of benign tumors among all the actual benign tumors.

For Class 1 (Malignant tumors)

- recall (1) measures the ability of the model to correctly identify instances of malignant tumors among all the actual malignant tumors.

		PREDICTED		
		No default (0 or N - Negative)	Default (1 - P - Positive)	
ACTUAL	No default (0 or N - Negative)	TN 4	Type I error 2 FP	6
	Default (1 - P - Positive)	FN 1 Type II error	3 TP	4
		5	5	0

$$\frac{\text{True Negatives}_0}{\text{True Negatives}_0 + \text{False Positives}_0}$$

$$\frac{\text{True Positives}_1}{\text{True Positives}_1 + \text{False Negatives}_1}$$

Recall in multi-class

	Predicted A	Predicted B	Predicted C
Actual A	20	5	2
Actual B	3	18	1
Actual C	1	2	15

Recall for each class (A, B, C):

- Recall for Class A:

$$\text{Recall}_A = \frac{20}{20+5+2} = \frac{20}{27}$$

- Recall for Class B:

$$\text{Recall}_B = \frac{18}{3+18+1} = \frac{18}{22}$$

- Recall for Class C:

$$\text{Recall}_C = \frac{15}{1+2+15} = \frac{15}{18}$$

overall recall using averaging methods

Micro-Averaged Recall

	Predicted A	Predicted B	Predicted C
Actual A	20	5	2
Actual B	3	18	1
Actual C	1	2	15

$$\frac{\text{Total True Positives}}{\text{Total True Positives} + \text{Total False Negatives}}$$

$$= \frac{20+18+15}{27+22+18}$$

```
# Calculate micro-averaged recall
micro_recall = recall_score(y_true, y_pred, average='micro')
```


overall recall using averaging methods

Macro-Averaged Recall

	Predicted A	Predicted B	Predicted C
Actual A	20	5	2
Actual B	3	18	1
Actual C	1	2	15

- Recall for Class A:

$$\text{Recall}_A = \frac{20}{20+5+2} = \frac{20}{27}$$

- Recall for Class B:

$$\text{Recall}_B = \frac{18}{3+18+1} = \frac{18}{22}$$

- Recall for Class C:

$$\text{Recall}_C = \frac{15}{1+2+15} = \frac{15}{18}$$

$$\frac{\text{Recall}_A + \text{Recall}_B + \text{Recall}_C}{3}$$

```
# Calculate macro-averaged recall
```

```
macro_recall = recall_score(y_true, y_pred, average='macro')
```

overall recall using averaging methods **weighted-Averaged Recall**

	Predicted A	Predicted B	Predicted C
Actual A	20	5	2
Actual B	3	18	1
Actual C	1	2	15

- Recall for Class A:

$$\text{Recall}_A = \frac{20}{20+5+2} = \frac{20}{27}$$
- Recall for Class B:

$$\text{Recall}_B = \frac{18}{3+18+1} = \frac{18}{22}$$
- Recall for Class C:

$$\text{Recall}_C = \frac{15}{1+2+15} = \frac{15}{18}$$

$$= \frac{3 \times \text{Recall}_A + 3 \times \text{Recall}_B + 3 \times \text{Recall}_C}{9}$$

```
# Calculate weighted-averaged recall
weighted_recall = recall_score(y_true, y_pred, average='weighted')
```

each class's contribution weighted by its proportion in the dataset.

Precision

- ratio of correctly predicted positive instances to the total number of instances predicted as positive.
- Precision is particularly relevant in scenarios where the cost of false positives (predicting positive when the actual class is negative) is relatively high.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

		PREDICTED		
		No default (0 or N - Negative)	Default (1 - P - Positive)	
ACTUAL	No default (0 or N - Negative)	TN 4	Type I error 2 FP	6
	Default (1 - P - Positive)	FN 1 Type II error	3 TP	4
		5	5	0

$$\text{Precision for Malignant Tumors} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

		PREDICTED		
		No default (0 or N - Negative)	Default (1 - P - Positive)	
ACTUAL	No default (0 or N - Negative)	TN 4	Type I error 2 FP 6	
	Default (1 - P - Positive)	FN 1 Type II error	3 TP 4	
		5	5	0

for - breast cancer data

In the context of the breast cancer dataset, precision would be an important metric for evaluating the performance of a classification model, particularly when considering the prediction of **malignant tumors** (positive class).

Positive Class (Class 1 - Malignant Tumors):

True Positives (TP): The number of instances where the model correctly predicted malignant tumors.

False Positives (FP): The number of instances where the model incorrectly predicted a tumor as malignant when it was actually benign.

	Predicted A	Predicted B	Predicted C
Actual A	20	5	2
Actual B	3	18	1
Actual C	1	2	15

Precision – multi class

provides insights into the reliability of positive predictions for each class in a multi-class classification scenario.

- Precision for Class A:**

$$\text{Precision}_A = \frac{20}{20+3+1} = \frac{20}{24}$$

- Precision for Class B:**

$$\text{Precision}_B = \frac{18}{5+18+2} = \frac{18}{25}$$

- Precision for Class C:**

$$\text{Precision}_C = \frac{15}{2+1+15} = \frac{15}{18}$$

Micro-Averaging:

$$\text{Micro-Averaged Precision} = \frac{20+18+15}{24+25+18}$$

Macro-Averaging:

$$\text{Macro-Averaged Precision} = \frac{\text{Precision}_A + \text{Precision}_B + \text{Precision}_C}{3}$$

Weighted-Averaging:

$$\text{Weighted-Averaged Precision} = \frac{3 \times \text{Precision}_A + 3 \times \text{Precision}_B + 3 \times \text{Precision}_C}{9}$$

	Predicted Benign	Predicted Malignant
Actual Benign	72	3
Actual Malignant	5	20

Precision and recall - balance

binary classification scenario where we're predicting whether a tumor is malignant (positive class) or benign (negative class).

Calculate precision and recall

- ❑ True Positives (TP): 20
- ❑ False Positives (FP): 5
- ❑ False Negatives (FN): 3
- ❑ True Negatives (TN): 72

		PREDICTED		
		No default (0 or N - Negative)	Default (1 - P - Positive)	
ACTUAL	No default (0 or N - Negative)	TN 72	Type I error FP 5	77
	Default (1 - P - Positive)	Type II error FN 3	TP 20	23
		75	25	0

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{20}{20 + 5} = \frac{20}{25}$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{20}{20 + 3} = \frac{20}{23}$$

27-08-2024

@ Copyright – bhupen@gridflowAI.com

34

mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension
12.050	22.72	78.75	447.8	0.06935	0.10730	0.07943	0.029780	0.1203	0.06659	...	12.57	28.71	87.36	488.4	0.08799	0.3214	0.29120	0.10920	0.2191	0.09349
14.870	20.21	96.12	680.9	0.09587	0.08345	0.06824	0.049510	0.1487	0.05748	...	16.01	28.48	103.90	783.6	0.12160	0.1388	0.17000	0.10170	0.2369	0.06599
9.668	18.10	61.06	286.3	0.08311	0.05428	0.01479	0.005769	0.1680	0.06412	...	11.15	24.62	71.11	380.2	0.13880	0.1255	0.06409	0.02500	0.3057	0.07875
13.280	13.72	85.79	541.8	0.08363	0.08575	0.05077	0.028640	0.1617	0.05594	...	14.24	17.37	96.59	623.7	0.11660	0.2685	0.28660	0.09173	0.2736	0.07320
13.340	15.86	86.49	520.0	0.10780	0.15350	0.11690	0.069870	0.1942	0.06902	...	15.53	23.19	96.66	614.9	0.15360	0.4791	0.48580	0.17080	0.3527	0.10160

Benign sample (negative /0)	Prob (0.35, 0.65)	Pred = benign (0)	TN
malignant sample (positive /1)	Prob (0.35, 0.65)	Pred = benign (0)	FN
malignant sample (positive /1)	Prob (0.15, 0.85)	Pred = benign (0)	FN
malignant sample (positive /1)	Prob (0.05, 0.95)	Pred = Malignant (1)	TP
malignant sample (positive /1)	Prob (0.31, 0.69)	Pred = benign (0)	FN

How would we get high precision?

We set a high threshold for **predicting malignancy**, meaning we only predict malignancy when we are extremely confident. (prob 0.9, else we predict it as **benign**)

Result is

- High cut off for the positive cases (malignant)
- Fewer **false positives** but may miss some malignant tumors (class =1)
- Means FNs are increasing

		PREDICTED		
		No default (0 or N - Negative)	Default (1 - P - Positive)	
ACTUAL	No default (0 or N - Negative)	TN 72	Type I error 5 FP	77
	Default (1 - P - Positive)	FN 3 Type II error	20 TP	23
		75	25	0

High →

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{20}{20 + 5} = \frac{20}{25}$$

Due to high cut off, TPs are confident, and FPs are also low

Low →

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{20}{20 + 3} = \frac{20}{23}$$

Many Positive (malignant, class=1) cases are now predicted as Class 0 (Benign) -> as the FNs are increasing

mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension
12.050	22.72	78.75	447.8	0.06935	0.10730	0.07943	0.029780	0.1203	0.06659	...	12.57	28.71	87.36	488.4	0.08799	0.3214	0.29120	0.10920	0.2191	0.09349
14.870	20.21	96.12	680.9	0.09587	0.08345	0.06824	0.049510	0.1487	0.05748	...	16.01	28.48	103.90	783.6	0.12160	0.1388	0.17000	0.10170	0.2369	0.06599
9.668	18.10	61.06	286.3	0.08311	0.05428	0.01479	0.005769	0.1680	0.06412	...	11.15	24.62	71.11	380.2	0.13880	0.1255	0.06409	0.02500	0.3057	0.07875
13.280	13.72	85.79	541.8	0.08363	0.08575	0.05077	0.028640	0.1617	0.05594	...	14.24	17.37	96.59	623.7	0.11660	0.2685	0.28660	0.09173	0.2736	0.07320
13.340	15.86	86.49	520.0	0.10780	0.15350	0.11690	0.069870	0.1942	0.06902	...	15.53	23.19	96.66	614.9	0.15360	0.4791	0.48580	0.17080	0.3527	0.10160

Benign sample (negative /0)	Prob (0.675, 0.325)	Pred = benign (0)	TN
malignant sample (positive /1)	Prob (0.35, 0.65)	Pred = malignant (1)	TP
Benign sample (negative /0)	Prob (0.56, 0.44)	Pred = malignant (1)	FP
Benign sample (negative /0)	Prob (0.05, 0.95)	Pred = Malignant (1)	FP
Benign sample (Negative /0)	Prob (0.64, 0.36)	Pred = Malignant (1)	FP

How would we get high recall?

We set a lower threshold (0.35) for predicting **malignancy**, making the model more inclusive.

Result is

- Low cut of (0.35) for the positive cases (malignant)
- More **false positives** but may miss some malignant tumors (class =1)
- Means FNs are increasing

		PREDICTED		
		No default (0 or N - Negative)	Default (1 - P - Positive)	
ACTUAL	No default (0 or N - Negative)	TN 72	Type I error 5 FP	77
	Default (1 - P - Positive)	FN 3 Type II error	20 TP	23
		75	25	0

Low →

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{20}{20 + 5} = \frac{20}{25}$$

Due to low cut off FPs are also high

High →

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{20}{20 + 3} = \frac{20}{23}$$

Many Positive (malignant, class=1) cases are now predicted
→ as the FNs are low

F1-score

Definition

- a measure of a model's accuracy on a dataset and is particularly useful when the classes are imbalanced.
- harmonic mean of precision and recall, providing a balance between the two.
- ranges from 0 to 1, with 1 indicating perfect precision and recall.

Precision

- ratio of correctly predicted positive observations to the total predicted positives.

Recall (Sensitivity)

- ratio of correctly predicted positive observations to the all observations in the actual class.

Calculation

		PREDICTED		
		No default (0 or N - Negative)	Default (1 - P - Positive)	
ACTUAL	No default (0 or N - Negative)	TN 4	Type I error 2 FP	6
	Default (1 - P - Positive)	FN 1 Type II error	3 TP	4
		5	5	0

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

	Predicted Positive	Predicted Negative
Actual Positive	40	10
Actual Negative	20	30

- Precision: $\frac{40}{40+20} = \frac{40}{60} = 0.67$
- Recall: $\frac{40}{40+10} = \frac{40}{50} = 0.80$

$$\text{F1-score} = 2 \times \left(\frac{0.67 \times 0.80}{0.67 + 0.80} \right)$$

$$\text{F1-score} = 2 \times \left(\frac{0.536}{1.47} \right)$$

$$\text{F1-score} = 2 \times 0.365$$

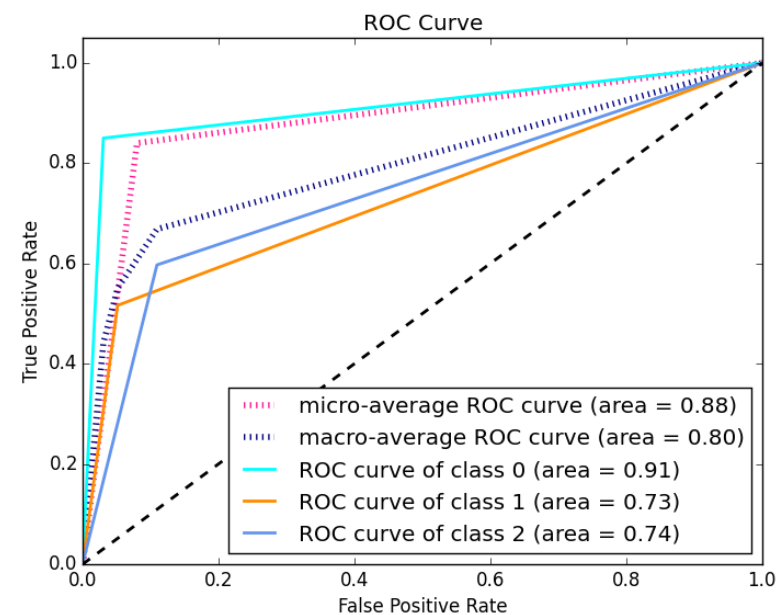
$$\text{F1-score} = 0.73$$

So, the F1-score is 0.73.

Receiver Operating Characteristic Curve (ROC)

Definition

- is a graphical plot that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied.
- curve is created by plotting the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings.



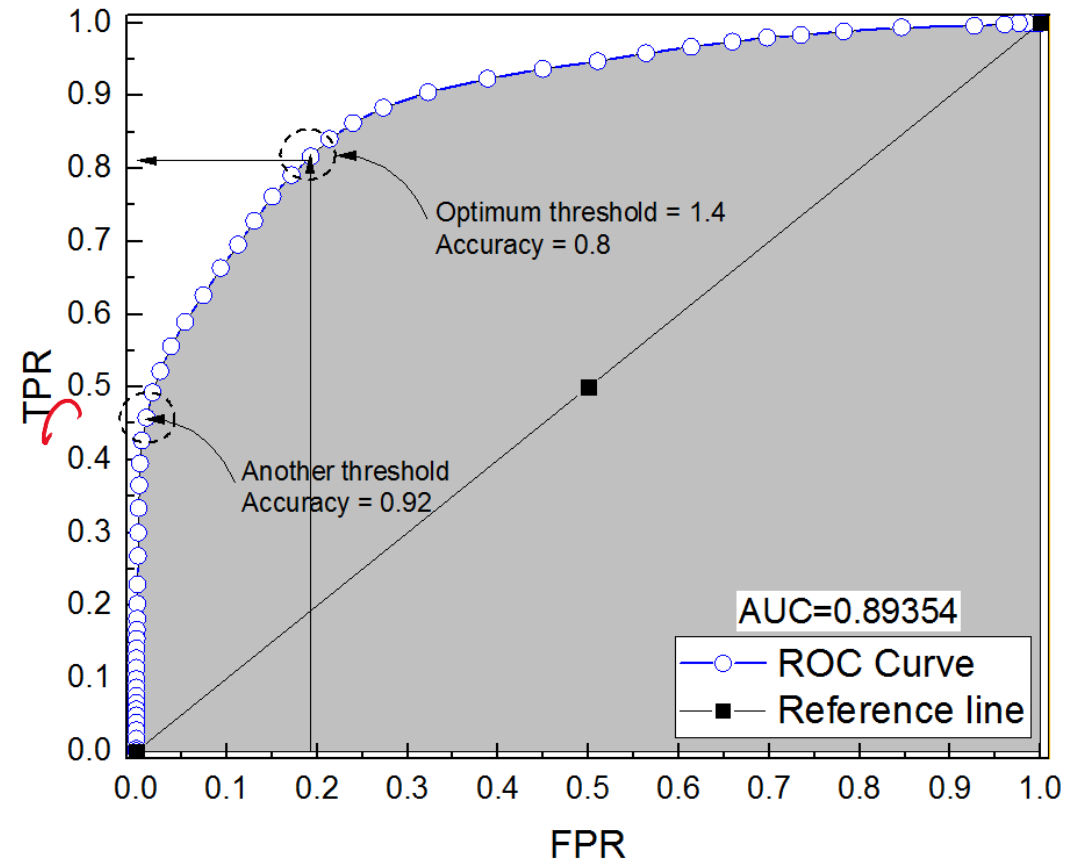
Intuition

- **True Positive Rate (TPR):** Also known as sensitivity or recall, it measures the proportion of actual positives that are correctly identified.

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **False Positive Rate (FPR):** Measures the proportion of actual negatives that are incorrectly identified as positives.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$



Example 1: Perfect Classifier

- Assume we have a binary classification problem where we are predicting whether an email is spam (positive class) or not spam (negative class).
- Let's construct a confusion matrix for a perfect classifier at different threshold values:

Confusion Matrix for a Perfect Classifier

- Suppose we have a dataset of 200 emails for testing:
 - 100 are spam (**positive** class)
 - 100 are not spam (**negative** class)
- At a certain threshold, the classifier makes predictions, and we get the following confusion matrix:

Actual / Predicted	Predicted Spam	Predicted Not Spam
Actual Spam	100 (TP)	0 (FN)
Actual Not Spam	0 (FP)	100 (TN)

- True Positives (TP): 100 (emails correctly predicted as spam)
- False Positives (FP): 0 (emails incorrectly predicted as spam)
- True Negatives (TN): 100 (emails correctly predicted as not spam)
- False Negatives (FN): 0 (emails incorrectly predicted as not spam)

Threshold 1: Low Threshold (say 0.2) (More Predictions as Spam)

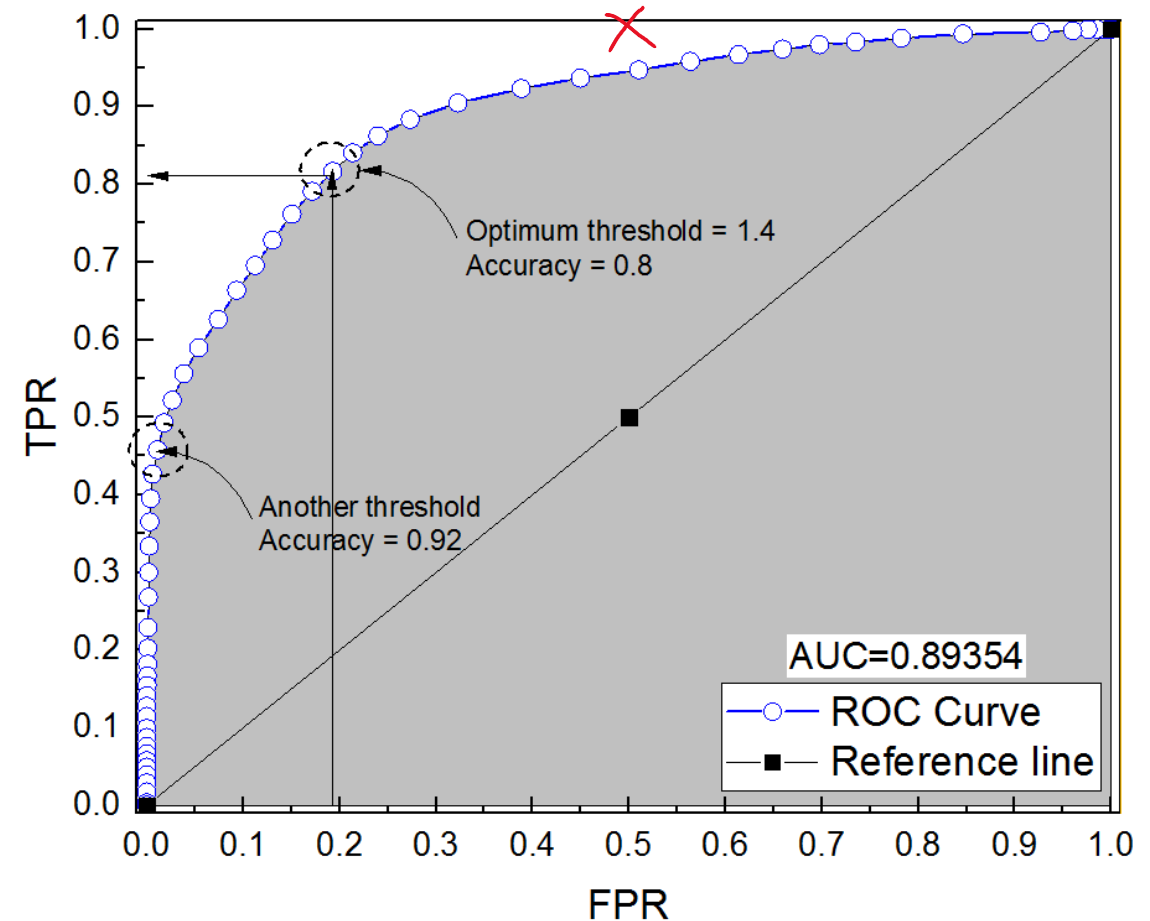
- **Threshold Setting**
 - Predict as spam if classifier's output ≥ 0.2
 - With this threshold, the classifier predicts 150 emails as spam
- **Results**
 - **True Positives (TP):** 100 (Actual spam emails correctly identified)
 - **False Positives (FP):** 50 (Not spam emails incorrectly classified as spam)
 - **True Negatives (TN):** 50 (Not spam emails correctly classified as not spam)
 - **False Negatives (FN):** 0 (No actual spam emails incorrectly classified)

Actual / Predicted	Predicted Spam	Predicted Not Spam
Actual Spam	100 (TP)	0 (FN)
Actual Not Spam	0 (FP) 50	100 (TN) 50

So...

- True Positive Rate (TPR): $\frac{100}{100+0} = 1.0$
- False Positive Rate (FPR): $\frac{50}{50+50} = 0.5$

This gives us a point on the ROC curve: (0.5, 1.0)



Threshold 2: Medium Threshold (say 0.5) (Balanced Predictions)

- **Threshold Setting:**

- Predict as spam if classifier's output ≥ 0.5
- With this threshold, the classifier predicts 100 emails as spam

- **Results**

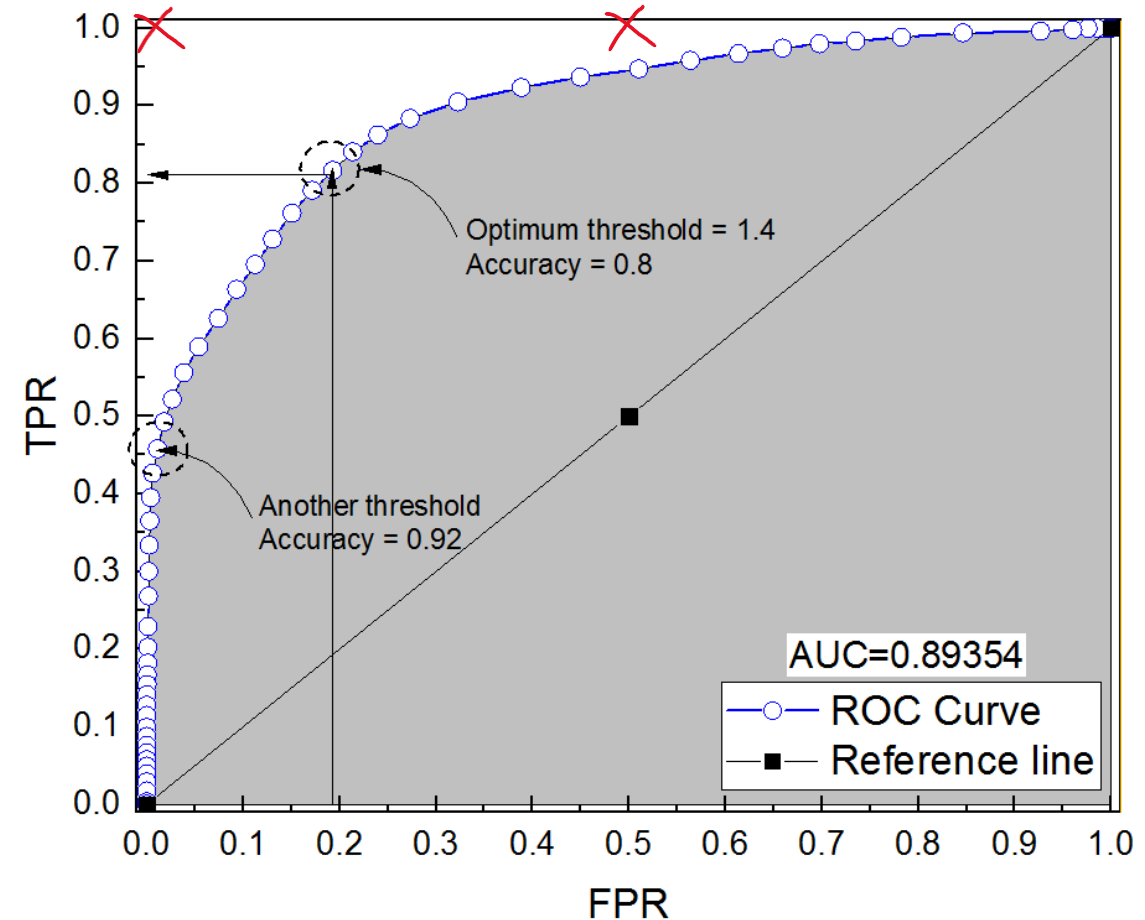
- True Positives (TP): 100 (Actual spam emails correctly identified)
- False Positives (FP): 0 (No not spam emails incorrectly classified as spam)
- True Negatives (TN): 100 (Not spam emails correctly classified as not spam)
- False Negatives (FN): 0 (No actual spam emails incorrectly classified)

Actual / Predicted	Predicted Spam	Predicted Not Spam
Actual Spam	100 (TP)	0 (FN)
Actual Not Spam	0 (FP)	100 (TN)

So ...

- True Positive Rate (TPR): $\frac{100}{100+0} = 1.0$
- False Positive Rate (FPR): $\frac{0}{0+100} = 0.0$

This gives us a point on the ROC curve: (0.0, 1.0)



Threshold 3: High Threshold (say 0.8) (Fewer Predictions as Spam)

- **Threshold Setting:**
 - Predict as spam if classifier's output ≥ 0.8
 - With this threshold, the classifier predicts only 20 emails as spam
- **Results**
 - **True Positives (TP):** 20 (Actual spam emails correctly identified)
 - **False Positives (FP):** 0 (No not spam emails incorrectly classified as spam)
 - **True Negatives (TN):** 80 (Not spam emails correctly classified as not spam)
 - **False Negatives (FN):** 80 (Actual spam emails incorrectly classified as not spam)

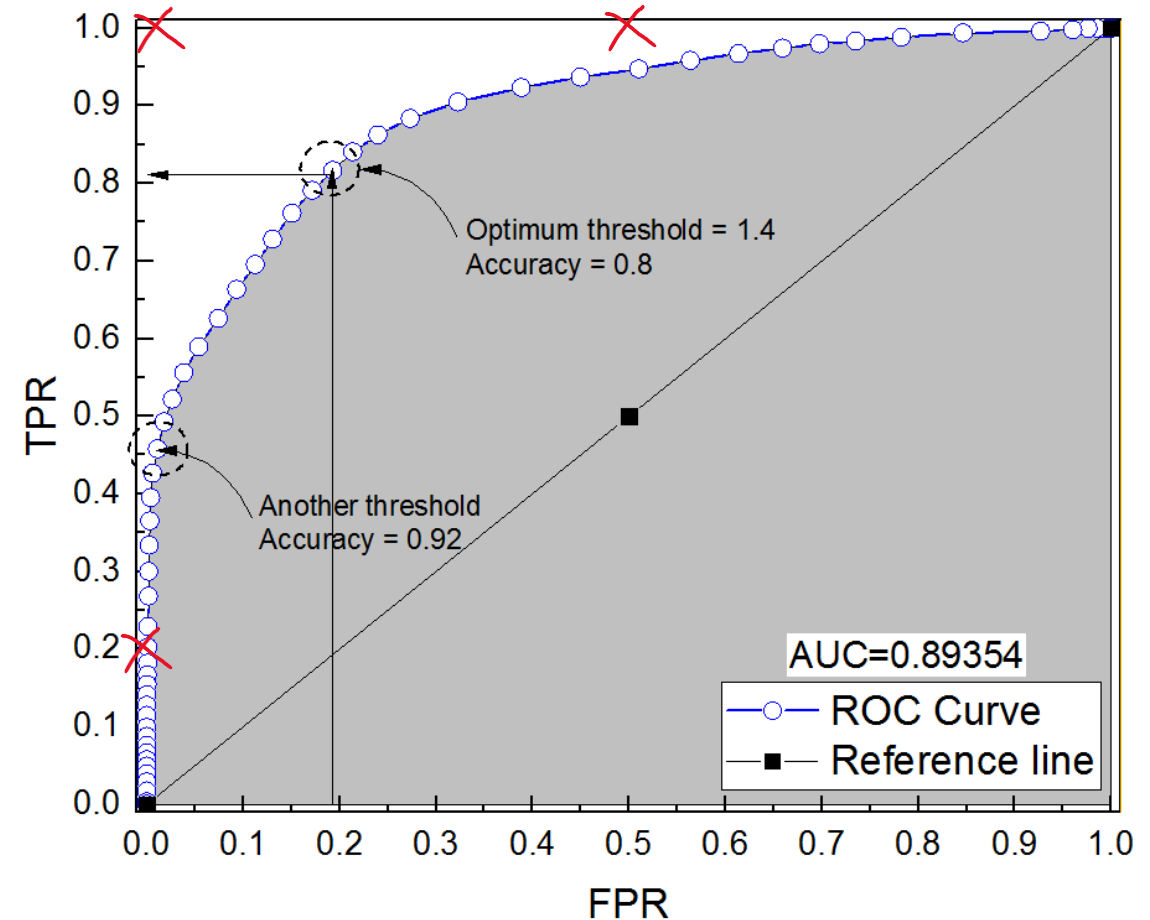
Actual / Predicted	Predicted Spam	Predicted Not Spam
Actual Spam	100 (TP) 20	0 (FN) 80
Actual Not Spam	0 (FP)	100 (TN) 80

So ...

- True Positive Rate (TPR): $\frac{20}{20+80} = 0.2$
- False Positive Rate (FPR): $\frac{0}{0+80} = 0.0$

This gives us a point on the ROC curve: (0.0, 0.2)

- (0.5, 1.0)
- (0.0, 1.0)
- (0.0, 0.2)



Characteristics of the ROC Curve for a Perfect Classifier

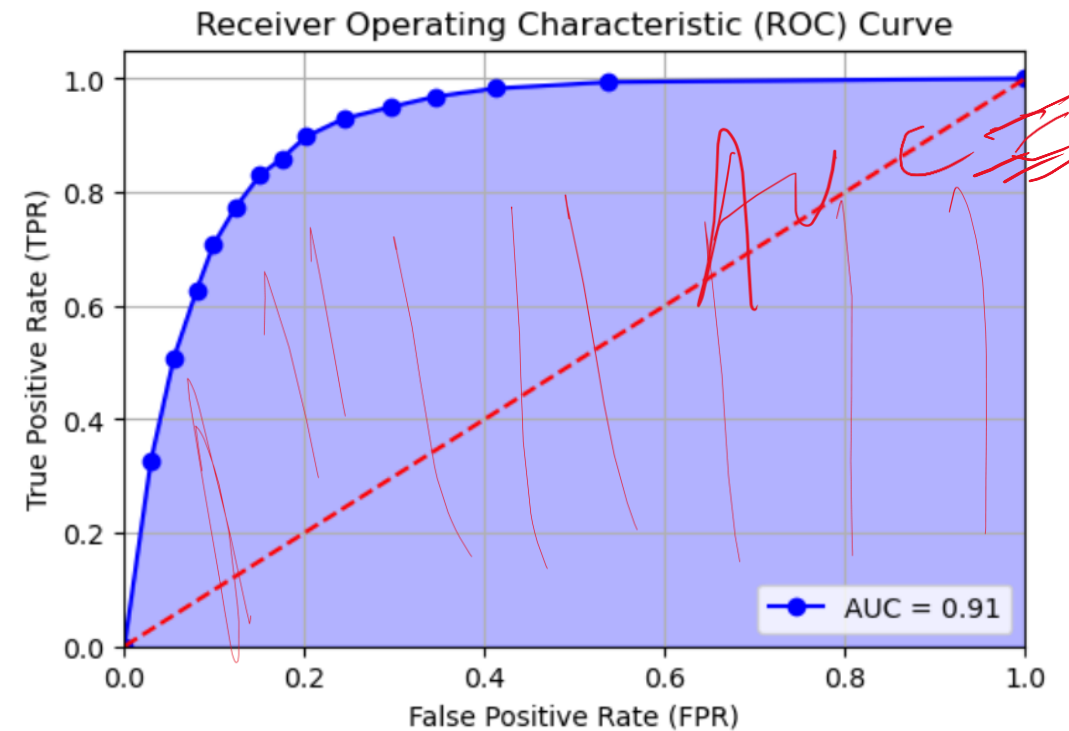
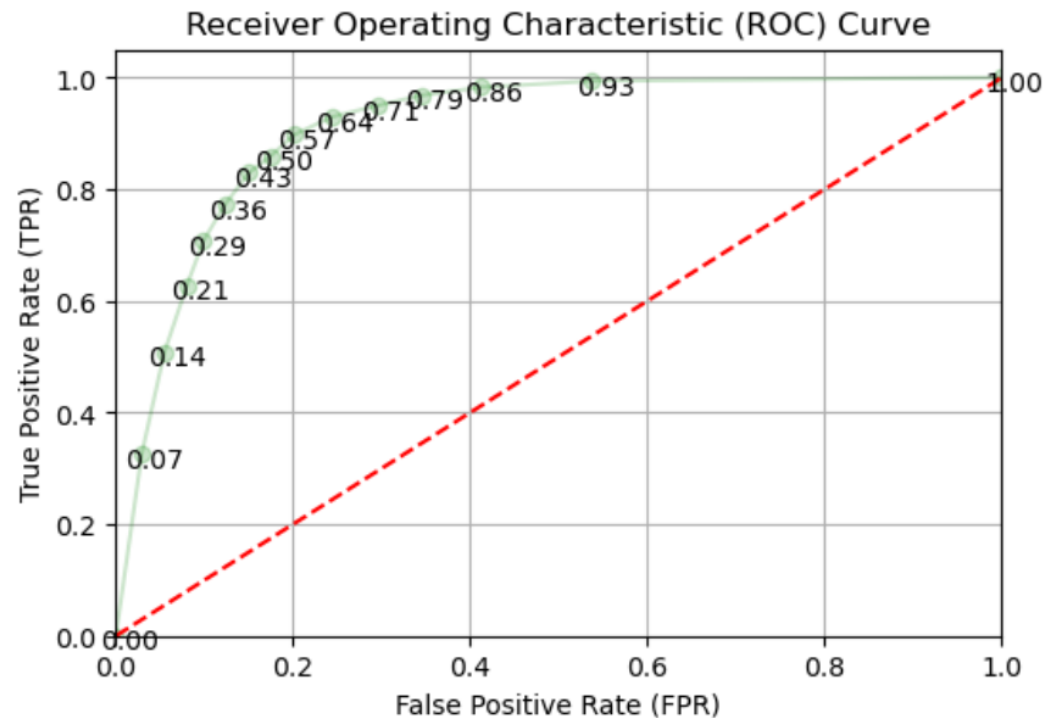


ROC curve for a perfect classifier is a line that starts from $(0.0, 0.0)$ and goes vertically up to $(0.0, 1.0)$, then horizontally to $(1.0, 1.0)$.

forms a perfect angular shape at the top-left corner of the ROC space, indicating perfect discrimination ability with no false positives at any threshold.

Conclusion

- changing the threshold affects the classifier's performance in terms of true positive and false positive rates.
- Adjustments can be made to model parameters or the model type for different scenarios or datasets.



Demo using
python/sklearn

(Accuracy, Precision,
recall, ROC-AUC)



53

27-08-2024



Let us detail out

Data Quality and Bias mitigation

Created for training and reference
purposes

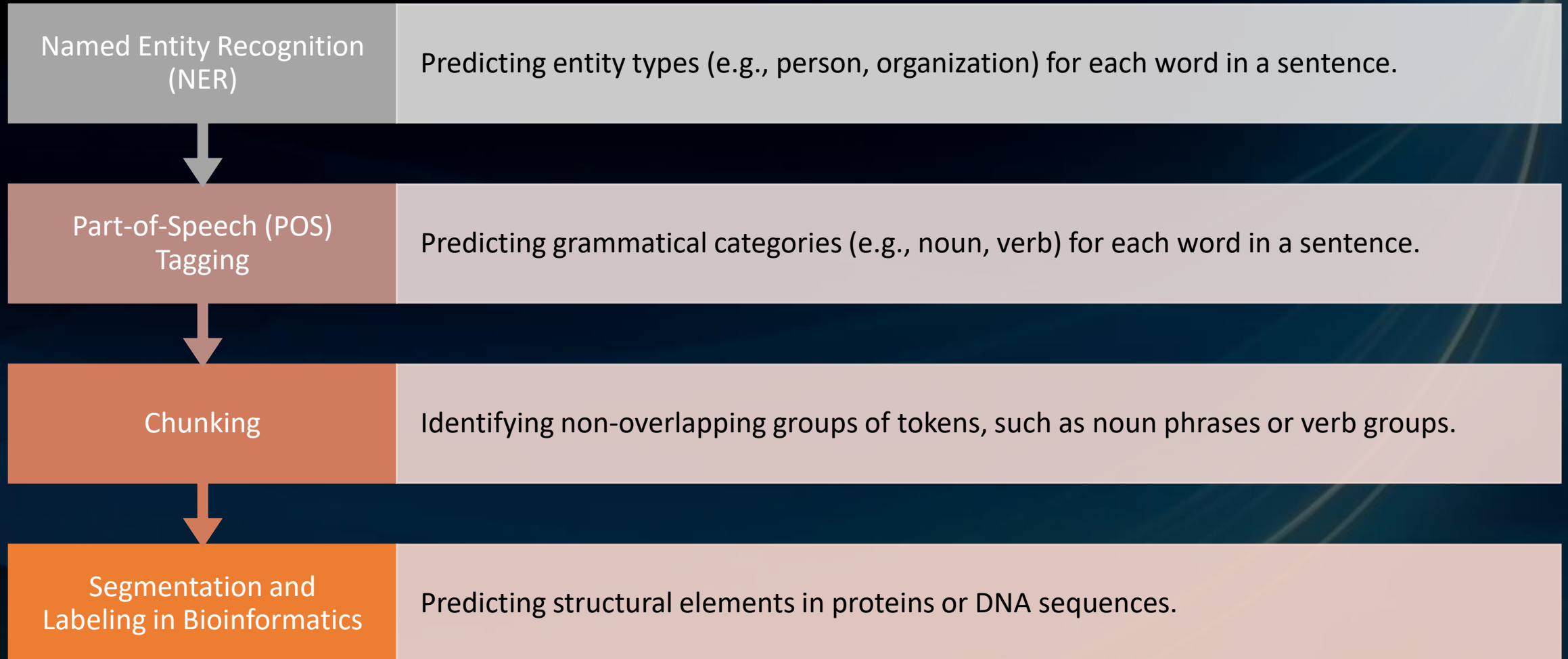
Created for training and reference
purposes



Sequence Labelling Metrics

- Sequence labeling metrics focus on comparing sequences of predicted labels with the sequences of true labels (ground truth).
- Unlike classification tasks where each instance is evaluated independently, sequence labeling tasks consider the order and coherence of predictions across multiple elements.

Examples



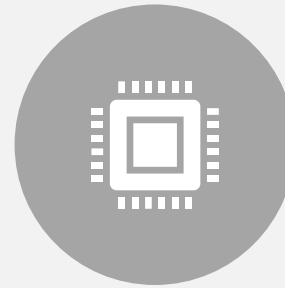
Language Generation Metrics

- **BLEU (Bilingual Evaluation Understudy Score)**: Measures the precision of n-grams in the generated text compared to a reference text.
- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)**: Measures the recall of n-grams, typically used in summarization.
 - **ROUGE-N**: Measures the overlap of n-grams.
 - **ROUGE-L**: Measures the longest common subsequence.
- **METEOR (Metric for Evaluation of Translation with Explicit ORdering)**: Considers synonyms, stemming, and word order.
- **CIDEr (Consensus-based Image Description Evaluation)**: Measures the consensus between generated and reference sentences, often used in image captioning.
- **TER (Translation Edit Rate)**: Measures the number of edits required to change the generated text into the reference text.
- **Perplexity**: Measures the uncertainty in predicting the next word in a sequence, commonly used in language models.

LLMs



Perplexity: is a measure of how well a language model can predict the next word in a sequence.



BLEU: BLEU (Bilingual Evaluation Understudy) is a metric for evaluating machine translation output.



ROUGE: ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is another metric for evaluating machine translation and summarization output.



METEOR: METEOR is a metric for evaluating machine translation output.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE is specifically used to evaluate the quality of summaries & translations generated by machines.

compares these automatic summaries to human-written summaries, judging how similar they are.

breakdown of ROUGE



Purpose - Assess quality of machine-generated **summaries** of text



How it works - Compares n-grams (sequences of n words) in the **machine summary** to n-grams in **human-written summaries**



Output - Score between 0 and 1, with a higher score indicating more similarity between the machine summary and the human-written one

Intuition



ROGUE measures the **overlap** of n-grams (typically unigrams, bigrams, and trigrams) between the **generated text** (candidate) and a **reference text** (usually a human-created summary).



The key idea is that a good summary will **share** many words and phrases with the reference summary.

Variants

ROGUE-N (ROGUE-n-gram)

- Measures the overlap of n-grams between the **candidate** and **reference** summaries.
- ROGUE-1 (unigram overlap), ROGUE-2 (bigram overlap), and ROGUE-L (longest common subsequence).

ROGUE-L (Longest Common Subsequence)

- Measures the length of the longest common subsequence of words between the candidate and reference summaries.
- useful for capturing sentence-level structure in addition to word overlap.

ROGUE-W (Weighted n-gram)

- Extends ROGUE-N by giving **different weights** to different n-grams

ROGUE-S (Skip-Bigram)

- Measures the overlap of skip-bigrams (pairs of words in their relative order but not necessarily consecutive) between the candidate and reference summaries.

Calculation

basic ROGUE-N formula for
precision, recall, and F-
measure

- **Precision:** $\text{Precision} = \frac{\text{Number of overlapping n-grams}}{\text{Total number of n-grams in candidate summary}}$
- **Recall:** $\text{Recall} = \frac{\text{Number of overlapping n-grams}}{\text{Total number of n-grams in reference summary}}$
- **F1-Score:** $F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Texts

Original Text	Reference Summary	Generated Summary
This is the full text or document that needs <u>to be summarized</u>. It contains all the information from which a summary will be generated.	This is a summary of the original text <u>created by humans</u>. It serves as the "<u>gold standard</u>" or benchmark for evaluating the quality of machine-generated summaries. The reference summary is considered to be a high-quality and accurate condensation of the original text.	This is the summary produced by a machine learning model or summarization algorithm. The generated summary is evaluated against the reference summary using ROGUE metrics.

Example to Illustrate

- **Original Text**

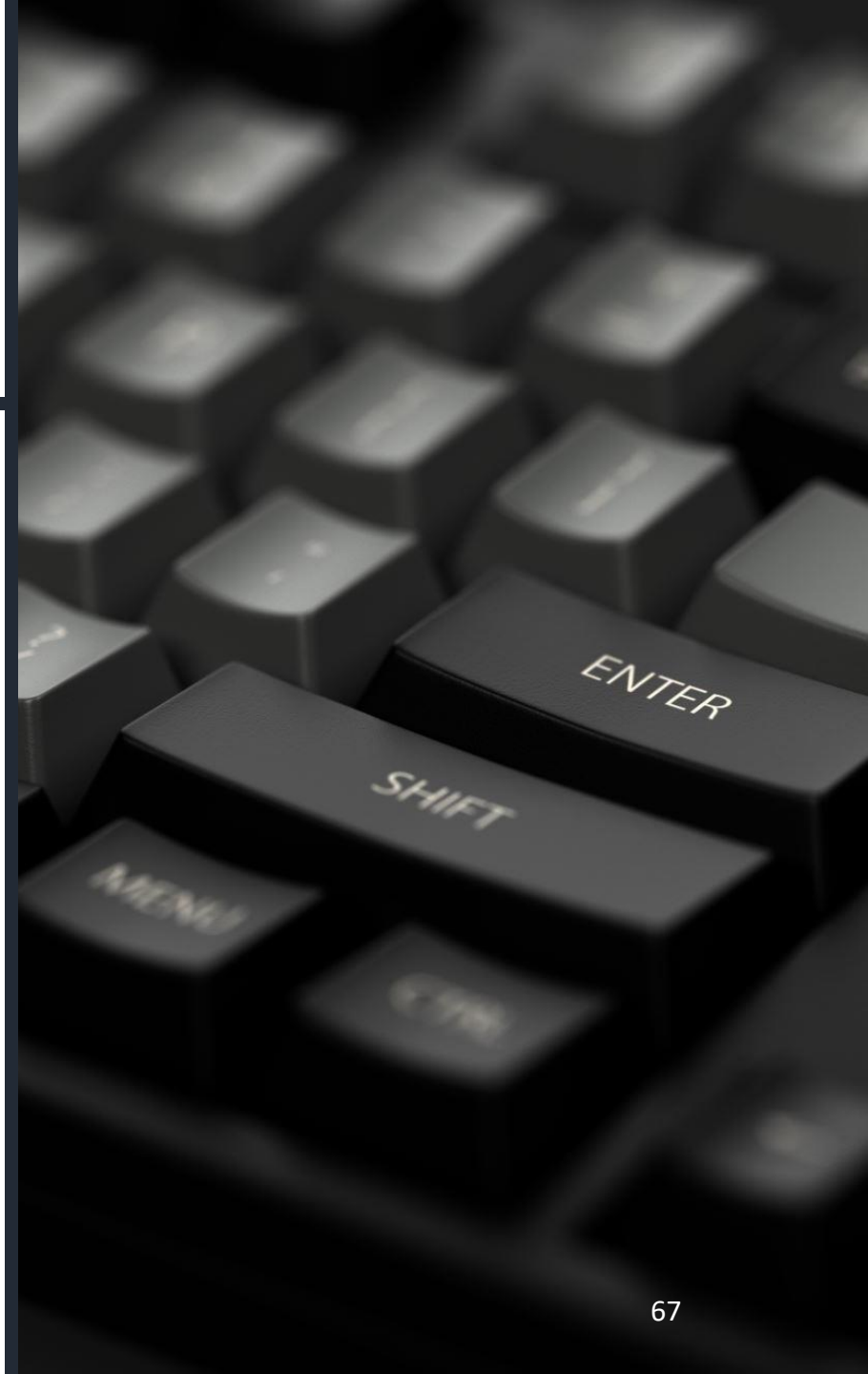
- "The quick brown fox jumps over the lazy dog. This phrase is often used to illustrate the use of all the letters in the English alphabet. It is a pangram and contains every letter from A to Z. Such sentences are useful for testing typing skills, font rendering, and keyboard layouts."

- **Reference Summary**

- "A quick brown fox jumps over a lazy dog, illustrating a pangram with all English letters, often used for testing typing and font rendering."

- **Generated Summary**

- "A quick brown fox jumps over the lazy dog, showing a sentence that contains every letter of the alphabet."



Sample

- **Reference** Unigrams:

- {"A", "quick", "brown", "fox", "jumps", "over", "a", "lazy", "dog", "illustrating", "pangram", "with", "all", "English", "letters", "often", "used", "for", "testing", "typing", "and", "font", "rendering"}

- **Generated** Unigrams:

- {"A", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog", "showing", "sentence", "that", "contains", "every", "letter", "of", "the", "alphabet"}

Example 1: ROGUE-1 (Unigram Overlap)



Original Text

"The quick brown fox jumps over the lazy dog."



Reference Summary

"A quick fox jumps over a lazy dog."

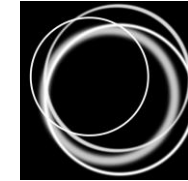
{"A", "quick", "fox", "jumps", "over", "a", "lazy", "dog"}



Generated Summary

"The fast brown fox leaps over the lazy dog."

{"The", "fast", "brown", "fox", "leaps", "over", "the", "lazy", "dog"}



Overlapping Unigrams

{"fox", "over", "lazy", "dog"}

- Precision: $\text{Precision} = \frac{\text{Number of overlapping n-grams}}{\text{Total number of n-grams in candidate summary}}$
- Recall: $\text{Recall} = \frac{\text{Number of overlapping n-grams}}{\text{Total number of n-grams in reference summary}}$
- F1-Score: $\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

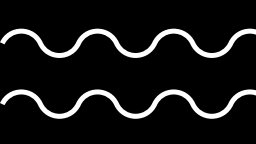
- Precision: $\frac{4}{9} = 0.44$
- Recall: $\frac{4}{8} = 0.50$
- F1-Score: $\frac{2 \times 0.44 \times 0.50}{0.44 + 0.50} = 0.47$

Example 2: ROGUE-2 (Bigram Overlap)

- Bigrams in Reference Summary: {"A quick", "quick fox", "fox jumps", "jumps over", "over a", "a lazy", "lazy dog"}
- Bigrams in Generated Summary: {"The fast", "fast brown", "brown fox", "fox leaps", "leaps over", "over the", "the lazy", "lazy dog"}
- Overlapping Bigrams: {"lazy dog"}

- Precision: $\text{Precision} = \frac{\text{Number of overlapping n-grams}}{\text{Total number of n-grams in candidate summary}}$
- Recall: $\text{Recall} = \frac{\text{Number of overlapping n-grams}}{\text{Total number of n-grams in reference summary}}$
- F1-Score: $\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

- Precision: $\frac{1}{8} = 0.125$
- Recall: $\frac{1}{7} = 0.14$
- F1-Score: $\frac{2 \times 0.125 \times 0.14}{0.125 + 0.14} = 0.13$



Example 3: ROGUE-L (Longest Common Subsequence)



ROGUE-L measures the longest common subsequence (LCS) between a candidate and reference text.

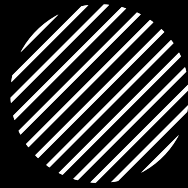


Unlike simple overlap metrics (e.g., precision or recall of n-grams), the LCS considers the order of words, making it more sensitive to the sequential nature of text.



The LCS is a sequence that can be derived from both the candidate and the reference text by deleting some characters without reordering the remaining characters.

Calculation of ROGUE-L



- **Identify the Longest Common Subsequence (LCS):**
 - The LCS of two sequences is the longest sequence that appears in both sequences in the same order but not necessarily consecutively.

2. Calculate Precision, Recall, and F1-Score:

- **Precision (P):** The ratio of the length of the LCS to the length of the candidate text.

$$P = \frac{\text{LCS}(X, Y)}{\text{Length of candidate (X)}}$$

- **Recall (R):** The ratio of the length of the LCS to the length of the reference text.

$$R = \frac{\text{LCS}(X, Y)}{\text{Length of reference (Y)}}$$

- **F1-Score:** The harmonic mean of Precision and Recall.

$$F1 = \frac{2 \times P \times R}{P + R}$$

Example

- Consider a candidate sentence X = "the cat sat on the mat" and a reference sentence Y = "the cat is sitting on the mat"
- The LCS between X and Y is "the cat on the mat" (ignoring "sat" in X and "is sitting" in Y)
- Length of LCS = 5 (words: "the", "cat", "on", "the", "mat")

Precision:

$$P = \frac{5}{6} = 0.833$$

Recall:

$$R = \frac{5}{7} = 0.714$$

F1-Score:

$$F1 = \frac{2 \times 0.833 \times 0.714}{0.833 + 0.714} = 0.769$$

Advantages of ROGUE-L

Order Sensitivity

Unlike unigram or bigram overlap, ROGUE-L takes the order of words into account, which is important for maintaining the coherence and readability of generated text.

Length Flexibility

It can handle texts of **different lengths** more gracefully by focusing on subsequences rather than contiguous n-grams.

Limitations of ROGUE-L

Complexity

Calculating the LCS can be computationally expensive for very long texts.

Partial Matching

may not fully capture semantic similarities if the exact words do not align, even if the overall meaning is preserved.

ROGUE-S



measures the overlap of skip-bigrams between the candidate text and the reference text.



A skip-bigram is any pair of words that appear in the same order in both texts, regardless of the number of words between them.



allows the metric to capture a broader range of meaningful relationships within the text that might be missed by strict consecutive n-gram measures.

Calculation of ROGUE-S

- **Identify Skip-Bigrams**
 - Generate all possible pairs of words that appear in the same order in both the candidate and reference texts.
- Calculate Precision, Recall, and F1-Score

Precision (P): The ratio of the number of matched skip-bigrams to the total number of skip-bigrams in the candidate text.

$$P = \frac{\text{Number of matched skip-bigrams}}{\text{Total skip-bigrams in candidate (X)}}$$

Recall (R): The ratio of the number of matched skip-bigrams to the total number of skip-bigrams in the reference text.

$$R = \frac{\text{Number of matched skip-bigrams}}{\text{Total skip-bigrams in reference (Y)}}$$

F1-Score: The harmonic mean of Precision and Recall.

$$F1 = \frac{2 \times P \times R}{P + R}$$

Example

- Consider a **candidate** sentence X = "the cat sat on the mat" and a **reference** sentence Y = "the cat is sitting on the mat".
- Skip-bigrams in X: ("the cat", "the sat", "the on", "the mat", "cat sat", "cat on", "cat mat", "sat on", "sat mat", "on mat").
- Skip-bigrams in Y: ("the cat", "the is", "the sitting", "the on", "the mat", "cat is", "cat sitting", "cat on", "cat mat", "is sitting", "is on", "is mat", "sitting on", "sitting mat", "on mat").
- Matched skip-bigrams: ("the cat", "the on", "the mat", "cat on", "cat mat", "on mat").

Precision:

$$P = \frac{6}{10} = 0.6$$

Recall:

$$R = \frac{6}{15} = 0.4$$

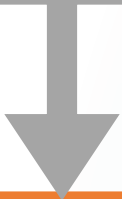
F1-Score:

$$F1 = \frac{2 \times 0.6 \times 0.4}{0.6 + 0.4} = 0.48$$

Advantages of ROGUE-S

Flexibility:

It can capture word relationships that are not immediately **adjacent**, allowing for a more contextually rich evaluation.



Order Sensitivity:

Maintains the order of words, which helps in assessing the coherence and fluency of the generated text.

Limitations of ROGUE-S

- **Complexity:**

- The number of skip-bigrams grows quadratically with the length of the text, which can make the computation intensive for longer texts.

- **Partial Matching:**

- Similar to ROGUE-L, it might not fully capture semantic similarities if exact word pairs do not align, even if the overall meaning is preserved.

Key points on ROUGE

Synonyms and Paraphrasing

- ROUGE primarily measures lexical overlap, which can be problematic because it doesn't account for synonyms or paraphrases.
- Two texts can convey the same meaning using different words, leading to low ROUGE scores despite high semantic similarity.



Idiomatic Expressions

- It struggles with idiomatic expressions and nuanced language that do not translate directly in terms of word overlap but have similar meanings.

Context and Coherence

Contextual Understanding

- ROUGE does **not** capture the coherence or the logical flow of the text.
- A summary that is a collection of relevant sentences may score high on ROUGE but can lack context and coherence.

Global Context

- ROUGE scores do not consider the broader context in which a piece of text appears, which is crucial for understanding the true meaning and relevance of the text.

Biases and Limitations



Bias Towards Extractive Summaries:

ROUGE tends to favor extractive summaries (where sentences are directly copied from the source) over abstractive ones (where new sentences are generated), even if the abstractive summaries are more coherent and fluent.



Overfitting to ROUGE:

Models optimized heavily for ROUGE scores may overfit to specific lexical patterns that maximize these scores rather than improving the overall quality and relevance of the generated text.

Recommendations

- **Complementary Metrics**
 - Use Multiple Metrics: Pair ROUGE with other evaluation metrics such as BLEU, METEOR, and BERTScore
 - Human Evaluation: Combine ROUGE scores with human evaluations
- **Adjust for Granularity**
 - Vary Granularity: Evaluate at different granularities (e.g., ROUGE-1, ROUGE-2, ROUGE-L) to capture unigram, bigram, and longest common subsequence overlaps.
 - Focus on Relevant Scores: For specific tasks, prioritize the ROUGE variants most relevant to the task's needs. For instance, ROUGE-L is useful for assessing fluency and coherence due to its focus on longest common subsequences.

Perplexity



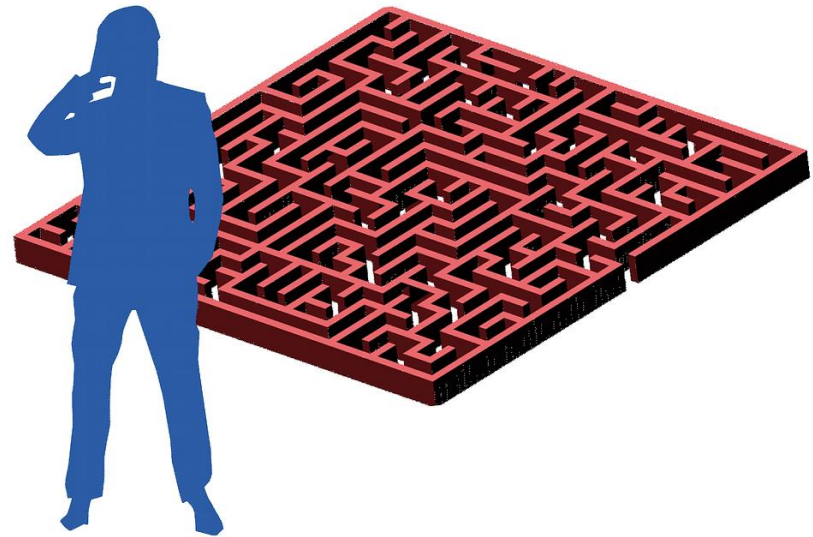
is a measure used to evaluate the performance of language models, including Large Language Models (LLMs).



provides a quantitative assessment of how well a language model predicts a sample of text.

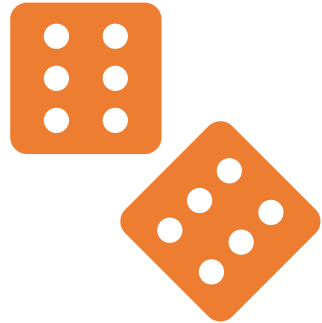


Lower perplexity values indicate better performance.

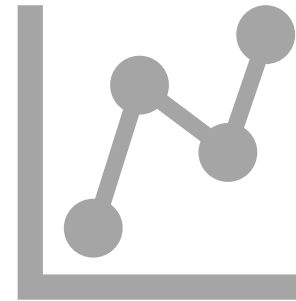


OpenAI has not publicly disclosed the specific perplexity values for its GPT (Generative Pre-trained Transformer) models, including GPT-3.

In other words



is a way to capture the degree of **'uncertainty'** a model has in predicting (i.e. assigning probabilities to) text.



is a measurement of how well a probability model predicts a sample.

Interpretation

Lower Perplexity:

- Indicates that the model is more **certain** about its predictions.
- A lower perplexity score suggests that the model is more **accurate** and makes better predictions.

Higher Perplexity:

- Indicates higher uncertainty in the model's predictions.
- A higher perplexity score means that the model is less accurate and less confident in its predictions.

Example

Test Sentence: "The cat is on the"

The model generates a probability distribution over the vocabulary for the next word in the sequence. Let's say the model assigns the following probabilities to three candidate words: "roof," "mat," and "moon."

Probability of "roof": 0.6

Probability of "mat": 0.3

Probability of "moon": 0.001

formula

$$\text{Perplexity} = \left(\frac{1}{P(w_1, w_2, \dots, w_N)} \right)^{\frac{1}{N}}$$

↑ high x.

Where $P(w_1, w_2, \dots, w_N)$ is the probability assigned by the model to the test sequence, and N is the number of words in the sequence.

In our example

$$(w_1, w_2, w_3, w_4) = 0.6 * 0.3 * 0.1$$

Let's assume N=4 (the number of words in the test sequence).

$$\text{Perplexity} = \left(\frac{1}{0.6 \times 0.3 \times 0.1} \right)^{\frac{1}{4}}$$

Example - 2

Suppose we have a simple language model trained on a small corpus with the following probabilities for the next word given a context:

- $P(\text{"cat"} \mid \text{"the"}) = 0.4$
- $P(\text{"dog"} \mid \text{"the"}) = 0.3$
- $P(\text{"runs"} \mid \text{"the"}) = 0.2$
- $P(\text{"sky"} \mid \text{"the"}) = 0.1$
- Now, let's evaluate the perplexity for the test sequence "the cat runs."

compute

1. Calculate Individual Probabilities:

1. $P(\text{"cat"}|\text{"the"}) = 0.4$
2. $P(\text{"runs"}|\text{"cat"}) = (\text{Assume, for simplicity, } P(\text{"runs"}|\text{"cat"})=0.5)$

2. Calculate Joint Probability of the Test Sequence:

1. $P(\text{"the cat runs"}) = P(\text{"cat"}|\text{"the"}) \times P(\text{"runs"}|\text{"cat"})$

3. Calculate Perplexity:

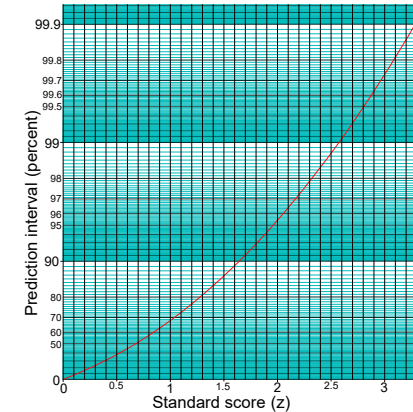
1. if $P(\text{"the cat runs"}) = 0.4 \times 0.5 = 0.2$, then

$$\text{Perplexity} = \left(\frac{1}{0.2}\right)^{\frac{1}{3}}$$

output range of values of perplexity



Lower Bound: minimum value of perplexity is 1, which would occur if the language model perfectly predicts the test data.



Upper Bound: no strict upper bound for perplexity. In practice, however, perplexity values can get quite large, especially for poorly performing models.

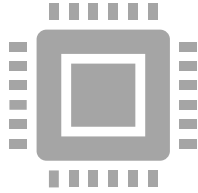
Application



Model Comparison:

Perplexity allows comparison of different language models.

The model with lower perplexity on a given test set is considered better because it predicts the data more accurately.



Hyperparameter Tuning:

In neural networks and other models used for language modeling, perplexity is often used as a metric during training to adjust hyperparameters (like learning rate, model architecture, etc.) for better performance.



Evaluation:

It serves as an evaluation metric in tasks like machine translation, where the model's ability to generate fluent and coherent translations can be assessed.

key points/facts/phrases related to perplexity in NLP

01

Perplexity Definition: measures how well a probability distribution or language model predicts a sample.

02

Calculation: calculated as the exponentiation of the entropy of the probability distribution over a test set.

03

High Perplexity: indicates that the language model is **less certain** about its predictions,

04

Type of Language Model: Neural language models typically achieve lower perplexity scores compared to traditional n-gram models

$$PP(W) = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i)}$$

...



High Perplexity: A high perplexity score indicates that the language model is less certain about its predictions, implying poorer performance in predicting the test set.



Relative Measure: Perplexity is relative because its absolute value can vary depending on factors like the size of the vocabulary and the complexity of the model architecture.



Type of Language Model: Neural language models typically achieve lower perplexity scores compared to traditional n-gram models due to their ability to capture complex dependencies in language.



Vocabulary Size: Increasing the vocabulary size typically increases perplexity, as the model has to distribute its probability mass over a larger set of words.

...



Influencing Factors: Perplexity in language models is influenced by factors such as model architecture, training data quality, and the effectiveness of smoothing techniques used.



Smoothing Impact: Smoothing techniques are used to adjust probability estimates for unseen or infrequent words, thereby potentially reducing perplexity by providing more accurate predictions.



Log Perplexity: Log perplexity is the natural logarithm of perplexity and is sometimes used in evaluation to simplify calculations and interpretability.



Inverse Metric: Perplexity is inversely related to model performance; lower perplexity scores indicate better language model performance.

...



Performance Indicator:

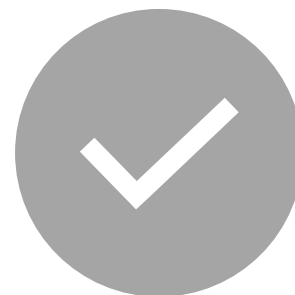
Smaller perplexity values signify that a language model makes more accurate predictions and is better at capturing the underlying structure of the language.



Domain Adaptation: Domain adaptation can affect perplexity, as models adapted to specific domains may perform better (lower perplexity) on test data from those domains.



Calculation Method: Perplexity for neural language models is commonly calculated using the test set data and the model's probability distribution over sequences.



Evaluation Metric: Perplexity is used in machine translation as an evaluation metric because it reflects how well the model predicts target language sequences given a source language input.

...



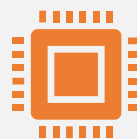
Out-of-Vocabulary (OOV): OOV refers to words in the test set that were not present in the training vocabulary, influencing perplexity calculations for those words.



Disadvantage: The primary disadvantage of perplexity as an evaluation metric is its sensitivity to factors like vocabulary size and the composition of the test set, which can limit its generalizability.



Fine-tuning Impact: Fine-tuning a pre-trained language model can improve perplexity by adapting the model to specific tasks or domains, thereby enhancing its predictive accuracy.



Model Complexity: Increasing model complexity can potentially reduce perplexity by allowing the model to capture more intricate patterns in the data, leading to better predictions.

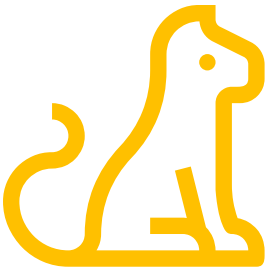
BLEU - Bilingual Evaluation Understudy

100



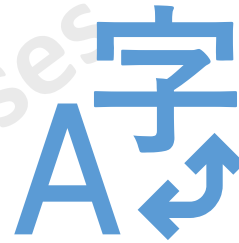
-
- is a metric used to evaluate the quality of machine-generated translations by comparing them to one or more human reference translations.
 - measures the precision of the generated translation against the reference translations by counting the number of overlapping n-grams between them.

example



Reference Translation:

"The cat is on the mat."



Candidate Translation:

"The cat is on the rug."

N-gram Precision

For unigrams (1-grams),

- "The," "cat," "is," "on," and "the" are common.
Precision = $5/6$.

For bigrams (2-grams),

- "The cat," "cat is," "is on," "on the," and "the rug" are common.
Precision = $5/5$.

For trigrams (3-grams),

- "The cat is," "cat is on," "is on the," and "on the rug" are common.
Precision = $4/4$.

For four-grams (4-grams),

- there are no common sequences.
Precision = 0.

Brevity Penalty

- In this example, the brevity penalty is 1 (since the lengths of the reference and candidate translations are equal).



$$\text{BLEU} = \text{Brevity Penalty} \times \exp \left(\frac{1}{4} \sum_{n=1}^4 \log \text{Precision}_n \right)$$

where Precision_n is the precision for the n-gram order.

BLEU Score Calculation

The BLEU score is the geometric mean of the precision scores for all the n-gram orders, with the brevity penalty included.

104

27-08-2024

In our case

- **BLEU Score:**

- $\text{BLEU} = 1 \times \exp \left(\frac{1}{4} \cdot (\log(5/6) + \log(5/5) + \log(4/4) + \log(0)) \right)$
- Since the logarithm of 0 is undefined, it is often treated as 0 in practice. Therefore, the BLEU score in this case would be 0.

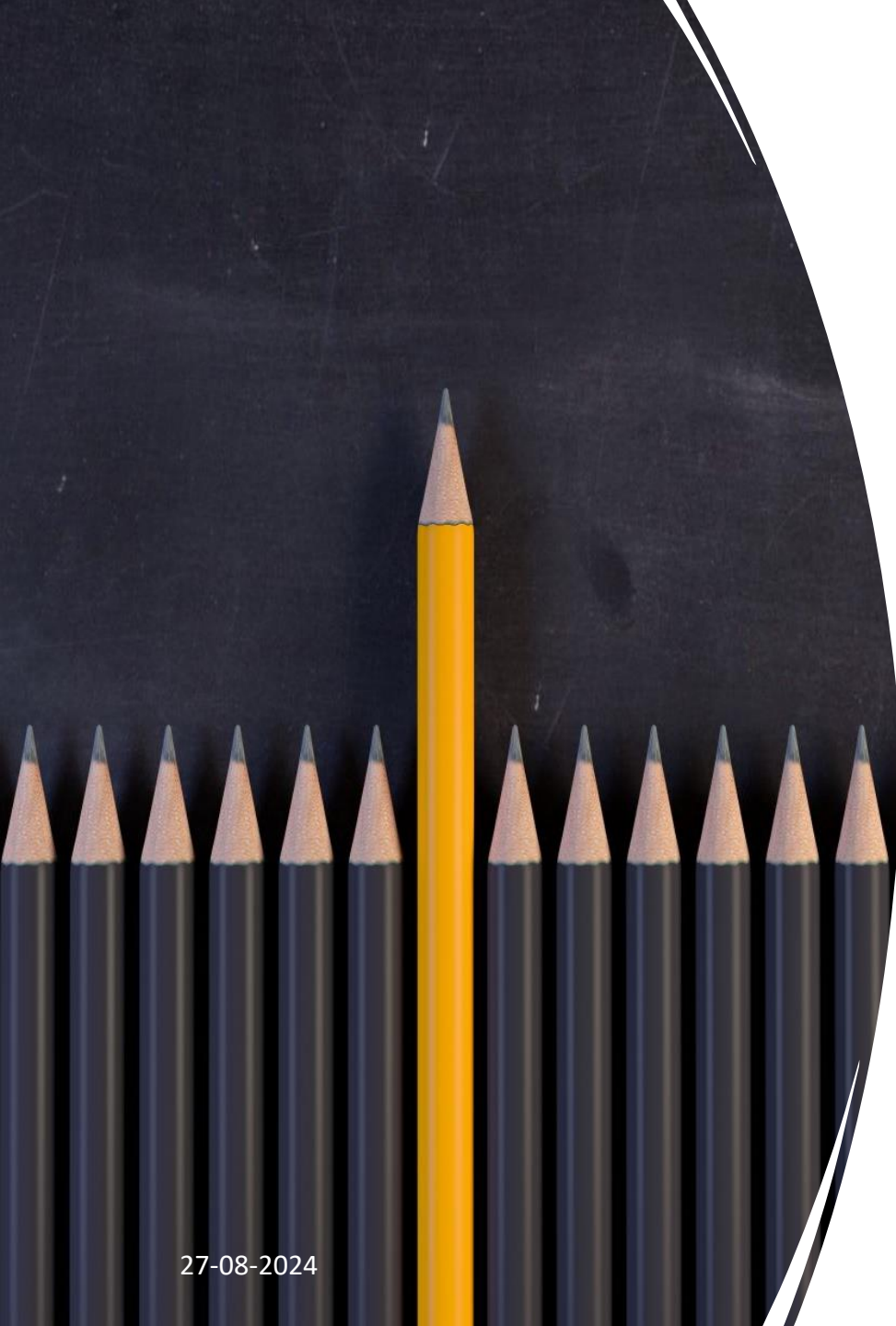
Examples

Suppose we have a machine translation system that produces a candidate translation and several human-generated reference translations

- Candidate:
 - "The cat is on the mat."
- References:
 - "There is a cat on the mat.",
 - "A cat is on the mat."

Compute

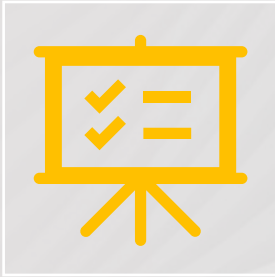
- precision for each n-gram order (typically up to 4).
- the geometric mean of these precisions.
- Apply brevity penalty to adjust for the length of the candidate compared to references.
- Multiply the brevity-penalized precision by the brevity penalty to get the final BLEU score.

A decorative graphic on the left side of the slide. It features a row of dark grey pencils standing upright. In the center of this row, one pencil is yellow and stands significantly taller than the others. The background behind the pencils is a dark, textured circular shape.

range

- BLEU scores **range from 0 to 1**, with higher values indicating better translation quality.
- It's important to note that BLEU has limitations, such as sensitivity to reference quality and the inability to capture semantic meaning comprehensively.

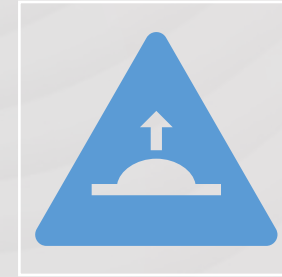
Range



Bleu Scores are between 0 and 1. A score of 0.6 or 0.7 is considered the best you can achieve.



Even two humans would likely come up with different sentence variants for a problem and would rarely achieve a perfect match.



For this reason, a score closer to 1 is unrealistic in practice and should raise a flag that your model is overfitting.



limitation

- BLEU is a Corpus-based Metric
 - The BLEU metric performs badly when used to evaluate individual sentences. For example, both example sentences get very low BLEU scores even though they capture most of the meaning.
 - Because n-gram statistics for individual sentences are less meaningful, BLEU is by design a corpus-based metric; that is, statistics are accumulated over an entire corpus when computing the score.
 - Note that the BLEU metric defined above cannot be factorized for individual sentences.

limitation

- No distinction between content and function words
 - The BLEU metric does not distinguish between content and function words, that is,
 - a dropped function word like "a" gets the same penalty as if the name "NASA" were erroneously replaced with "ESA".

limitation

Not good at capturing meaning and grammaticality of a sentence

- The drop of a single word like "not" can change the polarity of a sentence. Also, taking only n-grams into account with $n \leq 4$ ignores long-range dependencies and thus BLEU often imposes only a small penalty for ungrammatical sentences.

Normalization and Tokenization

- Prior to computing the BLEU score, both the reference and candidate translations are normalized and tokenized. The choice of normalization and tokenization steps significantly affect the final BLEU score.

Applying BLEU score



1. Machine Translation Research:

1. BLEU is widely used in research settings to assess the performance of machine translation systems. Researchers use BLEU scores to compare different models, techniques, or algorithms for translating text from one language to another.

2. Model Tuning and Optimization:

1. During the development and training of machine translation models, BLEU scores are often used as a metric for model tuning and optimization. Researchers and developers adjust parameters, architectures, or training data to improve BLEU scores.

3. Benchmarking Translation Systems:

1. BLEU is used as a benchmark metric for comparing different machine translation systems or services. It provides a standardized measure for assessing the quality of translations generated by different systems across various languages.

4. Evaluation in Competitions:

1. In machine translation competitions, BLEU scores are frequently employed as one of the evaluation metrics. Competitors aim to produce translations with higher BLEU scores to demonstrate the effectiveness of their translation systems.

5. Quality Assurance in Localization:

1. BLEU scores are used in the localization industry to assess the quality of translations for software, websites, and other content. Localization teams may use BLEU to monitor and improve the accuracy of translated materials.

6. Automated Evaluation in MT Services:

1. Online translation services often use BLEU scores for automated evaluation of the translations they provide. BLEU allows these services to quantify and communicate the quality of their translations to users.

Demo using python/sklearn (BLEU score metric)



113

27-08-2024

METEOR

Metric for Evaluation of Translation with Explicit Ordering



is an automatic metric for evaluating machine translation output that goes beyond simple word matching



Unlike metrics such as BLEU which primarily focus on exact word matches, METEOR aims to better align with human judgment



important features

- **Synonyms:**
- METEOR recognizes that different words can have similar meanings.
- includes a synonym matching module, where words with similar meanings are considered matches, even if they are not exact matches.
- typically achieved through the use of resources like [WordNet](#).

important features

- **Stemming**
- METEOR accounts for different morphological forms of a word.
- For example, "running" and "ran" would be considered equivalent because they share the same root form "run".
- Stemming helps to reduce the penalty for minor variations in word form that do not significantly change the meaning of a sentence.

important features

- **Precision and Recall**
- METEOR combines both precision & recall (harmonic mean)
- Precision : the proportion of matched words in the candidate translation that are also in the reference translation
- Recall : the proportion of matched words in the reference translation that are also in the candidate translation).
- balanced approach ensures that translations are rewarded for being both accurate and comprehensive

important features

Fragmentation Penalty

METEOR introduces a fragmentation penalty to account for the number of matched chunks.

The idea is that having many small chunks of matches (i.e., highly fragmented matches) is worse than having a few larger chunks, as it indicates that the order of words is quite different.



Example 1: Low Fragmentation Penalty

Reference Sentence	Candidate Translation	Matched Chunks	Fragmentation Penalty
• “The quick brown fox jumps over the lazy dog”	• “The quick brown fox jumps over the lazy dog”	• “The quick brown fox jumps over the lazy dog” • (1 chunk)	• Low, as the entire sentence is one continuous chunk.

Example 2: Moderate Fragmentation Penalty

Reference Sentence

- "The quick brown fox jumps over the lazy dog."

Candidate Translation

- "Quick the brown fox jumps over lazy the dog."

Matched Chunks

- "Quick" (1 chunk)
- "the brown fox jumps over" (2nd chunk)
- "lazy the dog" (3rd chunk)

Fragmentation Penalty

- Moderate, as the sentence is broken into three chunks, indicating some reordering.

Example 3: High Fragmentation Penalty

Reference Sentence

- "The quick brown fox jumps over the lazy dog."

Candidate Translation

- "Fox quick brown jumps the over lazy dog the."

Matched Chunks

- "Fox" (1 chunk)
- "quick brown" (2nd chunk)
- "jumps the" (3rd chunk)
- "over lazy" (4th chunk)
- "dog the" (5th chunk)

Fragmentation Penalty

- High, as the sentence is broken into five chunks, indicating significant reordering.

The fragmentation penalty is calculated using the formula

$$\text{Penalty} = 0.5 \times \left(\frac{c}{m}\right)^3$$

where:

- c is the number of chunks.
- m is the number of matched words.

Calculating Fragmentation Penalty

- $c = 8$
- $m = 8$

$$\text{Penalty} = 0.5 \times \left(\frac{8}{8}\right)^3 = 0.5 \times 1 = 0.5$$

- $c = 3$
- $m = 8$ (assuming all words match)

$$\text{Penalty} = 0.5 \times \left(\frac{3}{8}\right)^3 = 0.5 \times (0.375)^3 = 0.5 \times 0.052734375 = 0.0263671875$$

-
- how fragmentation penalty increases with more fragmented matches, indicating worse translation quality in terms of word order

overall METEOR score

$$\text{METEOR} = \left(\frac{10 \cdot P \cdot R}{R + 9 \cdot P} \right) \cdot (1 - \text{Penalty})$$

Limitations of METEOR

Complexity

- METEOR is more complex to implement and compute than simpler metrics like BLEU. It requires additional resources like synonym databases (e.g., WordNet) and stemming algorithms.

Language Dependency

- The effectiveness of METEOR's synonym and stemming modules can vary across different languages.

Fragmentation Sensitivity

- While the fragmentation penalty helps evaluate word order, it can sometimes overly penalize translations that make valid rephrasings or use different syntactic structures.

Handling of Paraphrases

- Although METEOR considers synonyms, it may still struggle with more complex paraphrases or idiomatic expressions that significantly alter the wording but not the meaning.

Demo using python/sklearn (METEOR)



125

27-08-2024



Translation Edit Rate (TER)

- is a metric used to evaluate the quality of machine translation by measuring the number of edits required to transform the generated (hypothesis) text into the reference text.
- edits can include insertions, deletions, substitutions, and shifts (reordering of words or phrases).

Created for training and reference
purposes

A circular portrait of a man with glasses and a mustache, wearing a dark shirt. The background of the portrait is a brick wall on the left and a teal wall on the right. The portrait is centered on a larger, light brown circular background.

Thanks !!